

Confindaid API — Integration Reference

<https://api.confindaid.com>

Generated on September 21, 2026

How it works



1. Create a credential

A client ID and secret, issued once from the previous step.



2. Exchange for a token

Trade them for a short-lived access token and a refresh token.



3. Call the API

Send the access token as a Bearer header on every request.

</> Integration code

Copy one of these into your backend. Each snippet exchanges the credential for an access token and then analyzes a piece of content — read the client id and secret from environment variables rather than hardcoding them.

cURL · analyze.sh

```
curl -X POST https://api.confindaid.com/v1/analyze \  
-H "Authorization: Bearer $CONFINAID_ACCESS_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"content": "Merhaba, sözleşme taslağını ekte gönderiyorum."}'
```

Node.js · confinaid.mjs

```
// 1. Exchange the credential for an access token (cache it for its Lifetime).  
async function getAccessToken() {  
  const res = await fetch("https://api.confindaid.com/v1/token", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({  
      client_id: process.env.CONFINAID_CLIENT_ID,  
      client_secret: process.env.CONFINAID_CLIENT_SECRET,  
    }),  
  });  
  if (!res.ok) throw new Error("Confindaid: token exchange failed");  
  const { access_token } = await res.json();  
  return access_token;  
}  
  
// 2. Analyze content.  
async function analyze(content) {  
  const res = await fetch("https://api.confindaid.com/v1/analyze", {  
    method: "POST",  
    headers: {  
      Authorization: `Bearer ${await getAccessToken()}`,  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({ content }),  
  });  
  if (res.status === 429) throw new Error("Confindaid: quota exhausted");  
  if (!res.ok) throw new Error(`Confindaid: ${res.status}`);  
  return res.json();  
}
```

```
const result = await analyze("Merhaba, sözleşme taslağını ekte gönderiyorum.");
console.log(result.is_risky, result.risk_score, result.findings);
```

Python · confinaid.py

```
import os
import requests

# 1. Exchange the credential for an access token (cache it for its lifetime).
def get_access_token() -> str:
    res = requests.post(
        "https://api.confinaid.com/v1/token",
        json={
            "client_id": os.environ["CONFINAID_CLIENT_ID"],
            "client_secret": os.environ["CONFINAID_CLIENT_SECRET"],
        },
        timeout=10,
    )
    res.raise_for_status()
    return res.json()["access_token"]

# 2. Analyze content.
def analyze(content: str) -> dict:
    res = requests.post(
        "https://api.confinaid.com/v1/analyze",
        headers={"Authorization": f"Bearer {get_access_token()}"},
        json={"content": content},
        timeout=30,
    )
    if res.status_code == 429:
        raise RuntimeError("Confindaid: quota exhausted")
    res.raise_for_status()
    return res.json()

result = analyze("Merhaba, sözleşme taslağını ekte gönderiyorum.")
print(result["is_risky"], result["risk_score"], result["findings"])
```

PHP · confinaid.php

```
<?php

// 1. Exchange the credential for an access token (cache it for its lifetime).
function confinaid_access_token(): string
{
    $ch = curl_init('https://api.confinaid.com/v1/token');
    curl_setopt_array($ch, [
        CURLOPT_POST => true,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_HTTPHEADER => ['Content-Type: application/json'],
        CURLOPT_POSTFIELDS => json_encode([
            'client_id' => getenv('CONFINAID_CLIENT_ID'),
            'client_secret' => getenv('CONFINAID_CLIENT_SECRET'),
        ]),
    ]);

    $body = curl_exec($ch);
    $status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);

    if ($status !== 200) {
        throw new RuntimeException('Confindaid: token exchange failed');
    }
}
```

```

    }

    return json_decode($body, true)['access_token'];
}

// 2. Analyze content.
function confinaid_analyze(string $content): array
{
    $ch = curl_init('https://api.confinaid.com/v1/analyze');
    curl_setopt_array($ch, [
        CURLOPT_POST => true,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_HTTPHEADER => [
            'Authorization: Bearer ' . confinaid_access_token(),
            'Content-Type: application/json',
        ],
        CURLOPT_POSTFIELDS => json_encode([
            'content' => $content,
        ]),
    ]);

    $body = curl_exec($ch);
    $status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);

    if ($status === 429) {
        throw new RuntimeException('Confindaid: quota exhausted');
    }
    if ($status !== 200) {
        throw new RuntimeException("Confindaid: {$status}");
    }

    return json_decode($body, true);
}

$result = confinaid_analyze('Merhaba, sözleşme taslağını ekte gönderiyorum.');
```

```

var_dump($result['is_risky'], $result['risk_score']);
```

Go · confinaid.go

```

package confinaid

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
    "os"
)

// AccessToken exchanges the credential for a token; cache it for its lifetime.
func AccessToken() (string, error) {
    payload, err := json.Marshal(map[string]string{
        "client_id": os.Getenv("CONFINAID_CLIENT_ID"),
        "client_secret": os.Getenv("CONFINAID_CLIENT_SECRET"),
    })
    if err != nil {
        return "", err
    }

    res, err := http.Post("https://api.confinaid.com/v1/token",
        "application/json", bytes.NewReader(payload))
    if err != nil {
        return "", err
    }
    defer res.Body.Close()
}
```

```

if res.StatusCode != http.StatusOK {
    return "", fmt.Errorf("confinaid: token exchange failed (%d)", res.StatusCode)
}

var out struct {
    AccessToken string `json:"access_token"`
}
if err := json.NewDecoder(res.Body).Decode(&out); err != nil {
    return "", err
}
return out.AccessToken, nil
}

// Analyze sends content for analysis.
func Analyze(content string) (map[string]any, error) {
    token, err := AccessToken()
    if err != nil {
        return nil, err
    }

    payload, _ := json.Marshal(map[string]string{
        "content": content,
    })

    req, _ := http.NewRequest(http.MethodPost,
        "https://api.confinaid.com/v1/analyze", bytes.NewReader(payload))
    req.Header.Set("Authorization", "Bearer "+token)
    req.Header.Set("Content-Type", "application/json")

    res, err := http.DefaultClient.Do(req)
    if err != nil {
        return nil, err
    }
    defer res.Body.Close()

    if res.StatusCode == http.StatusTooManyRequests {
        return nil, fmt.Errorf("confinaid: quota exhausted")
    }
    if res.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("confinaid: %d", res.StatusCode)
    }

    var out map[string]any
    return out, json.NewDecoder(res.Body).Decode(&out)
}

```

Java • Confinaid.java

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public final class Confinaid {

    private static final HttpClient CLIENT = HttpClient.newHttpClient();

    // 1. Exchange the credential for an access token (cache it for its lifetime).
    static String accessToken() throws Exception {
        String body = ""
            {"client_id": "%s", "client_secret": "%s"}
            """".formatted(
                System.getenv("CONFINAID_CLIENT_ID"),
                System.getenv("CONFINAID_CLIENT_SECRET"));
    }
}

```

```

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.confindaid.com/v1/token"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(body))
    .build();

HttpResponse<String> response =
    CLIENT.send(request, HttpResponse.BodyHandlers.ofString());

if (response.statusCode() != 200) {
    throw new IllegalStateException("Confindaid: token exchange failed");
}
// Parse with your JSON Library of choice.
return JsonParser.readString(response.body(), "access_token");
}

// 2. Analyze content.
static String analyze(String content) throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.confindaid.com/v1/analyze"))
        .header("Authorization", "Bearer " + accessToken())
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(
            "{ \"content\": \"" + content + "\" }"))
        .build();

    HttpResponse<String> response =
        CLIENT.send(request, HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 429) {
        throw new IllegalStateException("Confindaid: quota exhausted");
    }
    if (response.statusCode() != 200) {
        throw new IllegalStateException("Confindaid: " + response.statusCode());
    }
    return response.body();
}
}

```

C# · Confindaid.cs

```

using System.Net;
using System.Net.Http.Json;
using System.Text.Json;

public static class Confindaid
{
    private static readonly HttpClient Client = new();

    // 1. Exchange the credential for an access token (cache it for its lifetime).
    public static async Task<string> GetAccessTokenAsync()
    {
        var response = await Client.PostAsJsonAsync("https://api.confindaid.com/v1/token", new
        {
            client_id = Environment.GetEnvironmentVariable("CONFINAID_CLIENT_ID"),
            client_secret = Environment.GetEnvironmentVariable("CONFINAID_CLIENT_SECRET"),
        });

        if (!response.IsSuccessStatusCode)
            throw new InvalidOperationException("Confindaid: token exchange failed");

        var payload = await response.Content.ReadFromJsonAsync<JsonElement>();
        return payload.GetProperty("access_token").GetString()!;
    }

    // 2. Analyze content.

```

```
public static async Task<JsonElement> AnalyzeAsync(string content)
{
    using var request = new HttpRequestMessage(HttpMethod.Post, "https://api.confinaid.com/v1/analyze")
    {
        Content = JsonContent.Create(new { content }),
    };
    request.Headers.Authorization =
        new("Bearer", await GetAccessTokenAsync());

    var response = await Client.SendAsync(request);

    if (response.StatusCode == HttpStatusCode.TooManyRequests)
        throw new InvalidOperationException("Confinaid: quota exhausted");

    response.EnsureSuccessStatusCode();
    return await response.Content.ReadFromJsonAsync<JsonElement>();
}
```

Headers

Sent on every call to the API. The token endpoint takes only Content-Type — it is authenticated by the credentials in the body, not by a header.

Request

Header	Required	Description
Authorization	Required	Bearer <access_token> — the token from the exchange.
Content-Type	Required	application/json. Other types are rejected with 415.
X-Request-ID	Optional	Your own correlation id. Generated for you when omitted; capped at 128 chars of [A-Za-z0-9._-].

Response

Header	When	Description
X-Request-ID	Always	Correlation id for this call. Quote it in support tickets.
Retry-After	On 429	Seconds to wait before retrying once quota is exhausted.
WWW-Authenticate	On 401	Present on a 401, describing the authentication scheme the endpoint expects.

Endpoints

Authentication

POST

/v1/token

Exchange client credentials for an access token

^

Post your client ID and secret to the token endpoint. The access token is valid for one hour; refresh it rather than re-sending the secret.

Request

POST /v1/token

Content-Type: application/json

```
{
  "client_id": "cid_QbX_SoUTUbWMrUWhd2joxmFg",
  "client_secret": "csk_..."
}
```

200 • Response

```
{
  "access_token": "cat_...",
  "refresh_token": "crt_...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "analyze rewrite"
}
```

401 • Response

```
{
  "error": "invalid_client",
  "error_description": "Client authentication failed.",
  "request_id": "9f2c1e7b4a6d5083c1f9ab2d7e4c6081"
}
```

The token endpoints answer with the RFC 6749 §5.2 shape instead of the error envelope every other endpoint uses, so off-the-shelf OAuth2 client libraries can parse them. An unknown client_id and a wrong client_secret are reported identically, so the endpoint cannot be used to enumerate valid client ids. Transport-level failures (429, 5xx) still use the standard envelope.

POST

/v1/token/refresh

Exchange a refresh token for a new token pair



Refresh tokens are single-use — each refresh returns a new pair and invalidates the old one.

Request

POST /v1/token/refresh

Content-Type: application/json

```
{
  "refresh_token": "crt_..."
}
```

200 • Response

```
{
  "access_token": "cat_...",
  "refresh_token": "crt_...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "analyze rewrite"
}
```



Replaying an already-used refresh token is treated as a leak: every token for that credential is revoked, and you have to exchange the client secret again.

POST

/v1/token/revoke

Revoke an access or refresh token



Makes a token unusable immediately. Send either an access or a refresh token — the service works out which it is. Call this the moment a token leaks.

Request



```
POST /v1/token/revoke
Content-Type: application/json

{
  "token": "cat_..."
}
```

204 No Content

It always answers 204, including for a token that does not exist. Reporting "not found" would turn this endpoint into an oracle for checking whether a token is valid, so a 204 means "this token is not usable any more", not "a token was found and revoked". Revoking a refresh token does not revoke the access tokens already issued from it — revoke those too, or rotate the credential in the dashboard to invalidate everything at once.

Confinaid API

POST

/v1/analyze

Analyze content for risk



Analyze content and get back a risk score with the individual findings, positioned in the text.

Request



```
POST /v1/analyze
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json
X-Request-ID: 8f3c1b2a

{
  "content": "Merhaba, sözleşme taslağını ekte gönderiyorum."
}
```

200 • Response



```
{
  "request_id": "8f3c1b2a",
  "analysis_id": "c0ffee11-2233-4455-6677-8899aabbccdd",
  "is_risky": true,
  "risk_score": 0.82,
  "language": "tr",
  "findings": [
    {
      "start": 9,
      "end": 26,
```



```
    "label_id": "CF-TAX-V1-00068",
    "label_title": "Sözleşme",
    "label_path": "Ticari > Sözleşme",
    "quote": "sözleşme taslağı"
  }
],
"usage": { "remaining": -1 }
}
```

POST **/v1/rewrite** Rewrite content into a safer version



Rewrite content into a safer version. Pass an analysis_id to ground the rewrite in a previous analysis.

Request



```
POST /v1/rewrite
Authorization: Bearer <ACCESS_TOKEN>
Content-Type: application/json

{
  "content": "Merhaba, sözleşme taslağını ekte gönderiyorum.",
  "analysis_id": "c0ffee11-2233-4455-6677-8899aabbccdd"
}
```

200 • Response



```
{
  "request_id": "1d94ff70",
  "analysis_id": "c0ffee11-2233-4455-6677-8899aabbccdd",
  "rewritten_text": "Merhaba, sözleşme taslağını ekte paylaşıyorum.",
  "usage": { "remaining": -1 }
}
```

Errors



Every error uses the same envelope: a stable machine-readable code plus a human-readable message.

Error body



```
{
  "error": {
    "code": "quota_exceeded",
    "message": "Your usage quota for this operation has been exhausted.",
    "request_id": "9f2c1e7b4a6d5083c1f9ab2d7e4c6081"
  }
}
```

Status	Code	Handling	Meaning
400	invalid_request	Fix	The body is malformed or a required field is missing.
401	unauthorized	Fix	The access token is missing, invalid, expired or revoked.

Status	Code	Handling	Meaning
403	forbidden	 Fix	Your licence or the credential's scopes do not cover this operation.
403	feature_disabled	 Fix	The licence and scopes are fine, but the customer has switched this ca
402	payment_required	 Fix	No API pricing plan is configured for the account, so usage cannot be
404	not_found	 Fix	The referenced record does not exist. An analysis_id belonging to ano
405	method_not_allowed	 Fix	The endpoint exists but does not accept this HTTP method. All of ther
413	payload_too_large	 Fix	The request body exceeds 1 MiB.
415	unsupported_media_type	 Fix	Content-Type was not application/json.
429	rate_limited	 Retry	The request rate limit was exceeded. Transient — wait out Retry-After
429	quota_exceeded	 Fix	Your licensed quota is exhausted. Retrying will not fix it — it needs a p
500	internal_error	 Retry	Something failed on our side. Safe to retry with backoff.
501	not_implemented	 Fix	The operation is recognised but not yet available.
502	upstream_unavailable	 Retry	A dependency was unreachable. The request was not processed.
504	upstream_timeout	 Retry	A dependency took too long. The request was not processed.

New codes may be added over time. Branch on the HTTP status, treat an unrecognised code as its status implies, and never parse `message` — it is written for humans and can change without notice.

Tracing a request

Every call is tagged with a correlation id that follows it through the whole platform. It is the single fastest way to get an answer about one specific request — without it, support is searching by timestamp.

- 1 Send your own `X-Request-ID` if you already have a request or job id to tie this to. Omit it and one is generated for you.
- 2 You get it back in the `X-Request-ID` response header on every call, and again inside the body as `error.request\_id` when the call fails — so an error alone is enough to trace, even if you did not capture the headers.
- 3 Log it next to your own identifiers. When something goes wrong days later, this is what turns a vague report into an exact lookup.
- 4 Quote it in support tickets. With the id we can pull the exact request; without it we cannot.

Correlated request and its error response 

```
# Send your own id...
curl -i -X POST ${CONFINAID_API_URL}/v1/analyze \
  -H "Authorization: Bearer $CONFINAID_ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Request-ID: order-4417-retry-2" \
  -d '{"content": "..."}'
```

...and it comes straight back, on success and on failure alike.

HTTP/1.1 429 Too Many Requests

X-Request-ID: order-4417-retry-2

Retry-After: 3600

```
{"error":{"code":"quota_exceeded","message":"...","request_id":"order-4417-retry-2"}}
```



Ids you send are capped at 128 characters and restricted to letters, digits, dot, underscore and hyphen; anything else is stripped. Do not put personal data in them — the id is written to logs.